Technical Guide — **VeriSocial**

CSC1097 - Technical Guide — Dublin City University

Project Title: VeriSocial

Student 1 Name: Javier Alejandro Flores Baez — **ID Number**: 21748521
Student 2 Name: Joaquin Escober — **ID Number**: 21746259

Supervisor: Graham Healy

Table of Contents

Introduction Section 1 3
Context and Background 1.1 3
Motivation 1.2 3
Research Section 2 3
Theoretical Foundations 2.1 3
Design and Architecture Section 3 4
Module Breakdown 3.3 4
Fig.3.1 - VeriSocial Context Diagram 4
System Overview 3.1 5
Fig.3.2 - Architecture Overview Diagram 6
Fig.3.3 - Front-end Architecture Diagram 7
Design Decisions 3.2 7
Implementation Section 4 10
Technologies 4.1 10
Table.X.X - Frontend technologies list 10
Development Environment 4.2 11
Sample Code & Walkthrough Section 5 14
Key Code Snippets - (Frontend) 5.1 14
Key Code Snippets - (Backend) 5.2 16
Key Code Snippets - (AI Content Detection) 5.3 19
Key Code Snippets - (Deployment) 5.4 20
Problems Solved Section 6 21
Identified Issues 6.1 21
Strategies and Solutions 6.2 22
Results, Testing and Analysis Section 7 22
Testing Strategies 7.1 22
Fig.X.X - Sample view of Unit Tests 23
Future Work Section 8 24
Limitations 8.1 24
Potential Improvements 8.2 24
Long-term Vision 8.3 24

Abstract:

VeriSocial is a platform designed to combat the growing challenges of AI-generated content and bot activity in online communities. By using AI Content Detection Models and Bot Detection algorithms, we can aim to create a space for authentic human interaction. The project utilises anomaly detection techniques, such as Isolation Forest, One-Class SVM and Local Outlier Factor in order to identify suspicious bot-like behaviour. In addition to our anti-AI and Bot approach - we included gamified elements to raise user awareness about AI content. This document describes our development journey and the many tools used to aid in this project.

Introduction Section 1

Context and Background 1.1

In the digital modern landscape, online communities are becoming increasingly plagued with AI-generated content and automated bot interactions. This dulls the user experience, especially for users who aim to enjoy authentic human content and communication between real users. Our project, VeriSocial, aims to combat this growing issue by designing a platform that aims to filter out ai-generated content and detect and mitigate bot activity. Additionally by incorporating gamified elements into the platform that challenges users and increase engagement, we can increase their awareness regarding the subtleties of AI-generated content.

The following document outlines the technical guide for VeriSocial, detailing the rationale behind it, the research and supporting evidence and the systems overall design and architecture. The guide is intended to highlight the critical components that make up the project and the different design decisions that have led to the final product.

Motivation 1.2

The motivation for developing this project stems from technical, social and ethical issues in the contemporary digital world. Together such issues contribute to the decline of online spaces and thus the decline of the user's online experience.

- One of our main concerns / motivations for this project is the combatting of inauthentic online content and interactions. The introduction of AI content and automated bots has caused the decline in genuine and real interactions. In popular social media, this can be seen with fake bot accounts, spam and fake advertisements. All of which drive artificial engagement, one that is not human. It is this type of issues that have lead to the beliefs of ideas such as the Dead Internet Theory [LINK]
- We wanted to not only add mitigating measures against these concerns, but also educate users on these topics to increase their awareness and familiarise themselves with the nuances of AI-generated content. By engaging users in these topics, we can take the initiative and aid users to become more informed and knowledgeable, allowing them to be able to distinguish between real vs fake in digital platforms.

Research Section 2

Theoretical Foundations 2.1

With the basis of our project revolving around the idea of anti-AI and bots, key areas of interest / research include anomaly detection techniques. These techniques are concerned with the process of identifying rare, or

strange events that raise suspicion as they can differ significantly from the majority of the observed data. In the context of our project, these techniques will be used to “detect” bots within our platform. Many studies show that bots typically show abnormal behaviour such as unusually high activity levels and abnormal interaction patterns. [LINK] [LINK]. By measuring activity quantitatively - anomaly detection becomes a natural fit for this premise.

One such anomaly detection method we incorporated is the use of **Isolation Forest** algorithms [LINK]. A distance-based algorithm that, unlike traditional methods, isolates anomalies by recursively partitioning the data using random values or “hyperplanes”. The key idea being that anomalies are “few” and “different” and therefore, when graphed and measured, are more spread out from the collection of normal data or the majority of it. During the random partitioning, isolated or anomalous data are isolated much faster than normal points, offering a more efficient computation. Using Isolation Forests gives us a more scalable and efficient approach, making it suitable for large datasets, such as those used in Social Media. [LINK] [LINK].

By adopting these techniques in our project, we can demonstrate and thereby support the idea of filtering out AI content and bot interactions within the VeriSocial platform.

Design and Architecture Section 3

Module Breakdown 3.3

The **Context Diagram** below illustrates a high level overview of the different key modules of VeriSocial and its various data flows. These modules communicate with each other, passing data and returning data for processing and viewing.

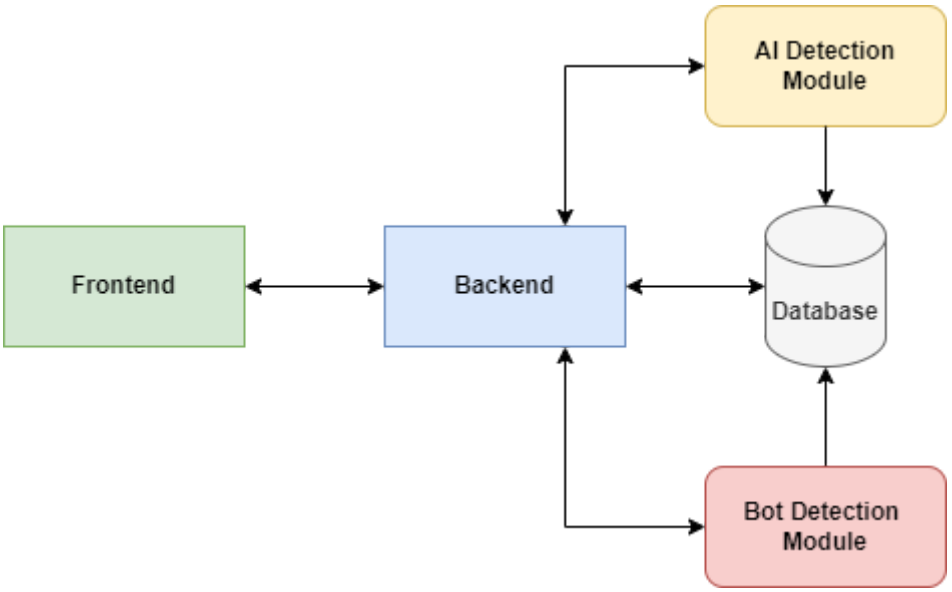


Fig.3.1 - VeriSocial Context Diagram.

Frontend: This is where user interaction occurs. It serves as a presentation layer and its main responsibilities are the passing and receiving of data from the backend. This data is used for display and processing of user data such as posts and activity.

Backend: Acts as the central hub of our system, it processes requests, handles logic and coordinates communication with not only the frontend but also the AI detection and the Bot detection module. It also communicates with the Database to store and retrieve user data.

AI Detection Module: Image and text data is passed on to this module from user posts. It utilises machine learning models to calculate the likelihood of posts being AI-generated. The result is then returned to the backend for processing.

Bot Detection Module: User activities are passed onto this module as vectors. This module utilises various algorithms, (Isolation Forest, One-Class SVM and Local Outlier Factor) to detect anomalies based on the overall collected user database. Results are returned to the backend for processing.

Database: Stores key information such as user data, posts, activity logs, and AI results. Many of the key modules communicate with the Database.

System Overview 3.1

The following **Architecture Overview Diagram** (AOD) serves as a more detailed representation of our Project's structure. Much like the Context Diagram above - we can see how the modules and and their different sub-components communicated with each other.

Most of the communication is done via HTTP requests and SQL queries with the Databases. **Nginx** serves as a proxy to handle and manage incoming HTTP requests and forwards them to the Backend services. To process these calls, the Backend sends HTTP requests to the various modules for AI and bot detection.
Another important factor to note is that our system uses Cloud Infrastructure for hosting. In this case **DigitalOcean**. Each module is hosted on their own droplet. The main reasons for this is Fault Isolation and Resource Optimisation. Having each service run on its own droplet (virtual server) allows us to minimise the impact of potential failure. If one module crashes or experiences issues, the entire system will not break down. It also allows for easier maintenance and debugging. Resource optimisation also becomes key as the different AI Detection modules can be CPU intensive. Hosting them on separate droplets allows for the ability to fine tune resource allocation for each AI module.

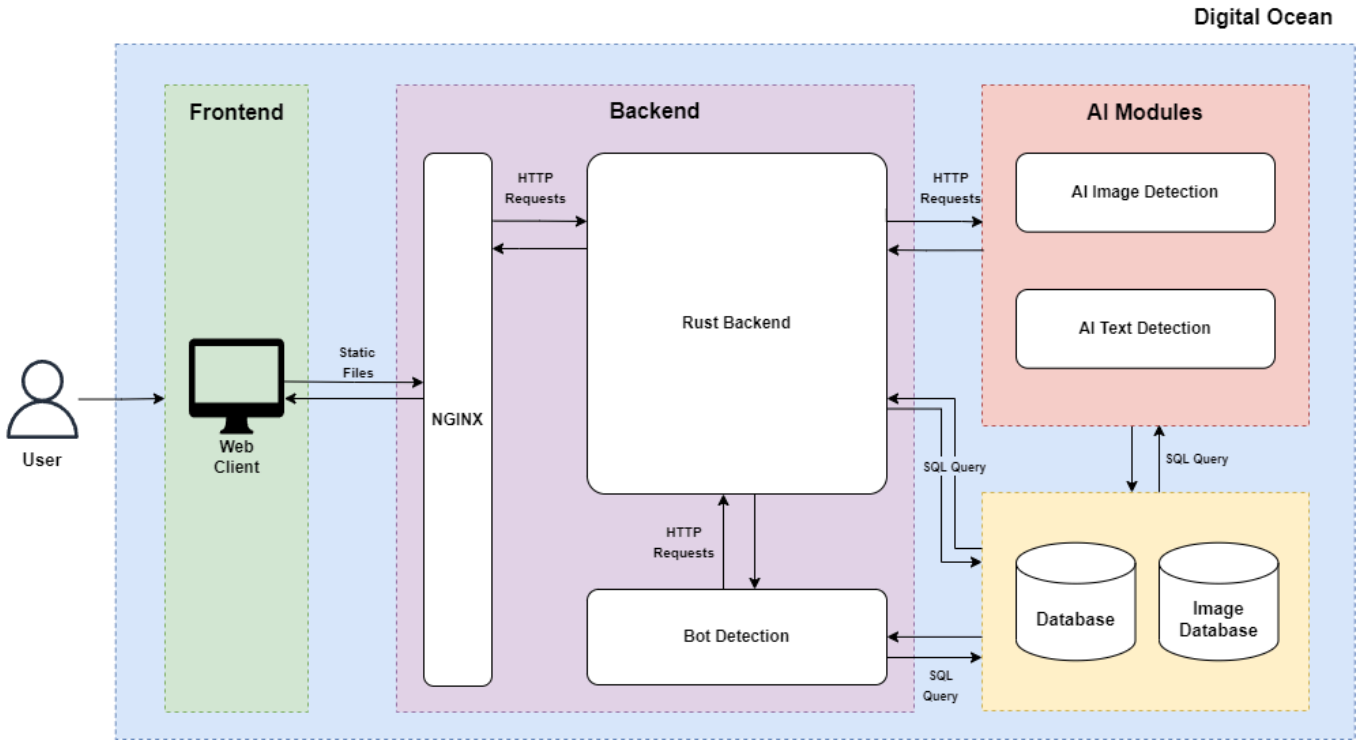


Fig.3.2 - Architecture Overview Diagram

Front End Diagram

Below is a simplified overview of how data (or states) is accessed and updated in the frontend. State management is handled via the React **Redux** libraries. States are stored in the "Store". The store is a collection of states in which components can be accessed globally. Reducer in redux simply updates these collections of states. For a component to update a state it must first dispatch an "Action". These actions are then observed by the **Redux-Saga** middleware - triggering a set of operations once detected. The middleware is used for handling side-effects and asynchronous calls. API calls are handled in the middleware and from there the response data is sent to the reducer for state updating.

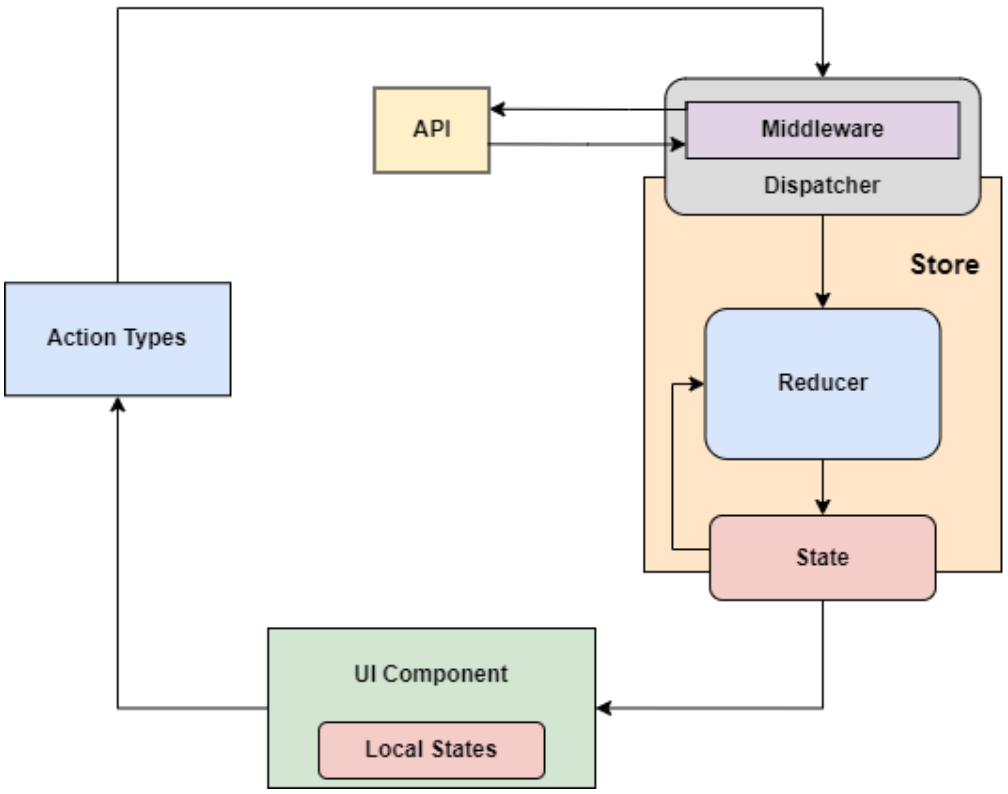


Fig.3.3 - Front-end Architecture Diagram.

Back-End Design Focus

The backend utilizes the repository design pattern in order to abstract and "mock" all functions that require to call another server (in most cases the database). The abstract layer creates some buffer that allows us to create business logic independent of the SQL logic, keeping them separate. The benefits are that the repositories are able to be mocked, and the business logic is able to be unit tested. All while, the tests for the repositories can be separated into integration testing. More on this in section 5.

Design Decisions 3.2

Below is a list of chosen technologies, showcasing their benefits and why we used them for this project. The following is a high-level overview of the key components used for both the front-end and back-end of the project.

Front-End design decisions

Framework and Bundler: **React** + **Vite**

React offers a modular architecture with its dynamic component model. This component-based architecture allows development to become easier and maintainable via reusable UI code. The bundler, Vite, complements React by providing fast and optimised development experience:

- Minimal configuration and support for JSX / TSX and CSS modules, allows us to spend less time on webpack-style setup.

- Faster cold starts by using native ECMAScript (standardisation specification that JavaScript follows for browser support). Vite serves source files on demand instead of pre-building the entire app, allowing for almost instant launches.

Component Library: **Chakra UI**
ChakraUI provides a comprehensive set of pre-styled UI components, ranging from form inputs, modals and notifications. By using ChakraUI we can focus on designing and implementing VeriSocial instead of spending more resources on re-inventing basic UI elements. Built-in theming utilities allows us to easily apply VeriSocial's brand colors and font styling as well.

State Management: **Redux**
Redux allows for easy global state management via centralising application state and storing them in a single "store". This makes our data flows straightforward and understandable. Combining this with Redux-Saga and Redux-Persist - we can create scalable and modular code and allow easy handling of critical data within the browser's local storage.

Front-end Testing: **Vitest**
For unit and integration testing, we use Vitest (with JSDOM). On top of being compatible with the Vite bundler, Vitest allows us to run tests considerably faster than most popular React testing libraries, such as Jest, and requires little to no configuration.

Back-End design decisions

Language Choice: **Rust**
Rust is an up and coming language with a strong emphasis on speed and strict typing. Despite being relatively new, its ecosystem is rising exponentially especially within the field of web development. This is because it compiles to binaries comparable with C/C++.

Framework: **Actix-Web**
Actix-web is a web-framework built for the rust ecosystem. It was chosen due to its high ranking in terms of performance among the other Rust web-frameworks. It's built on top of the Actix actor library and leverages Tokio (Rust's most famous asynchronous library) for efficient, non-blocking I/O. This allows for creation of type safe APIs that are robust and performant.

Database: **MySQL**
We wanted to utilize a relational database because of its robustness that we needed. We wanted the ability to create transactions in case some parts of our requests needed to be rolled back.

AI and Bot Detection decisions

Language Choice: **Python**
Python contains a rich ecosystem of packages, which gives our project access to well-tested implementations of statistical and machine-learning algorithms. For our strategy of bot detection we decided to quantify user activity into user vectors. This included the frequency of posting, liking and commenting. By using this

quantitative approach, we can easily integrate and feed them to different types of anomaly detection algorithms that use numerics as their inputs.

Packages: **Scikit-Learn**
Alongside our implementation for the Isolation Forest algorithm, we opted to use algorithms / estimators offered in scikit-learn's package. We utilised One-Class SVM and Local Outlier Factor from scikits, with each adhering to the same process as our Isolation Forest implementation (using user vectors). This allowed us to cover for some of the limitations with Isolation Forest while also mitigating single-model biases.

AI Model Wrapper: **Transformers (Sentence Huggers)**

Deployment decisions

Host **DigitalOcean**
DigitalOcean was chosen for deployment as it was an easy alternative compared to AWS, as it did not have a per minute compute price and instead held a per-month price. This gave us reassurance that we would not be spending too much in our deployment (in case we mess up)

Technology **Docker-Compose**
Docker as a containerization engine is amazing, and allowed us to work on deployment ready scenarios within our own laptops. The original idea for the project was intended to host all modules on a docker service but there were some issues along the way.

Technology **System services**
The system services for those modules that were not able to be deployed using docker used system services. This allowed us to go away from the issues we were facing with docker. More on this in problems and solutions

Implementation Section 4

Technologies 4.1

Below are the main technologies used in the frontend aspect of VeriSocial. Most of the frontend code is composed of **JavaScript**.

Technology	Version	Purpose / Description
React	v18.3.1	JavaScript library for building web applications.
Vite	v6.0.5	Bundler for React.
ChakraUI	v2.10.4	Component Library for React.
Redux Toolkit	v2.5.0	React State management library.
Redux-Saga	v1.3.0	Middleware for handling side effects. (e.g. asynchronous calls or API Calls).
Redux-Persist	v6.0.0	Library for persisting redux states to browser storage.
Axios	v1.7.9	Promised based HTTP client for API requests and handling responses.
React Router DOM	v7.1.1	Provides navigation and routing capabilities to handle pages.
Vitest	v3.0.4	Unit Testing Framework for Vite.
React Testing Library	v16.2.0	Testing utility for testing user interactions and component behaviour.
ESLint	v9.17.0	Static code analysis tool. Used to enforce coding standards.

Table.4.1 - Frontend technologies list

Below are the main technologies used in the backend aspect of VeriSocial. Most of the backend code consists of Rust.

Technology	Version	Purpose / Description
actix-web	v4.10.2	Web framework for Rust that focuses on concurrency. For API and Web services.
actix-cors	v0.7	Cross-Origin Resource Sharing (CORS) support for safe resource sharing.
serde	v1.0.219	Framework for serializing and deserializing data in Rust.
sqlx	v0.8.3	Database connection and query library for SQL & MySQL support.
reqwest	v0.12.11	Rust HTTP client use for requests to external APIs or servers.
argon2	v0.5.3	Password hashing library that implements Argon2 algorithm.

aws-sdk-s3	v1.82.0	Official AWS SDk for S3 storage services and integration with DigitalOcean.
tokio	v1.44.2	Asynchronous runtime for Rust for Actix concurrency and I/O operations.
dotevny	v0.15.7	For loading environment variables and simplifying configuration management.

Table.4.2 - Backend technologies list

Below are the main technologies used in the bot detection and AI detection aspect of VeriSocial. The code in this module consists of Python.

Bot Detection

Technology	Version	Purpose / Description
Scikit-learn	v0.24.2	Machine Learning Library for python. We used models here for our bot detection module. Models included One-Class SVM and Local Outlier Factor
MySqlConnector-python	v8.0.23	MySQL driver for Python. Allows Python Programs to be able to interact and connect with MySQL databases.

Table.4.3 - Bot Detection technologies list

AI Content Detection

Technology	Version	Purpose / Description
FastAPI	v0.115.12	The server framework utilizes an asynchronous model in python.
Transformers	v4.51.3	Sentence-transformers for the classification models.
MySqlConnector-python	v9.3.0	MySQL driver for Python. Allows Python Programs to be able to interact and connect with MySQL databases.
Pillow	v11.0.0	Library for opening images into memory in python.
Torch	v2.6.0	Open-Source ML framework (was needed by transformers).
Pydantic	v2.11.3	A library used for data validation and parsing.
boto3	v1.37.37	The AWS sdk kit for python.

Table.4.4 - AI Content Detection technologies list

Development Environment 4.2

For ensuring seamless and collaborative workflow during development, we employed the following technologies and software to aid us in our project.

Atlassian Jira

Purpose: Used for project management via task creation, planning and monitoring. Planned features of VeriSocial would first be logged as a Ticket in the backlog. Each developer would be responsible for creating and detailing the ticket requirements. Once a ticket is created, we create a feature branch for that ticket in

gitlab with structured branch names that link to the ticket number. E.g. tickets are in the form of "**VS-XX**", and feature branches would be named by "**VS-XX-feature-branch**".

Gitlab

Purpose: Serves as our version control system (VCS) for managing source code and Continuous Integration and Continuous Delivery (CI/CD) pipelines. Gitlab allows the developers to track code changes, perform code reviews and streamline our testing and deployment process via pipelines. Completed merge requests (MR) would also automatically close the related ticket in Jira.

DigitalOcean

Purpose: Secure Shell (SSH) was used for accessing and managing DigitalOcean droplets. Droplets are used to deploy all of our services. We are able to remotely log onto DigitalOcean servers to configure, maintain and deploy our services on the cloud infrastructure. This was done to ensure secure and reliable services.

Visual Studio Code

Purpose: Open-source code editor used to develop, and test our code. VSC as a work environment is compatible with both the frontend and backend coding languages as well as the AI modules, making it suitable for use by both project developers.

Docker

Purpose: Used to containerise the development environment, ensuring consistency across various stages of development, testing and deployment. Dockers allows developers to simulate environments and isolate dependencies, making it particularly useful for running services like the Database and AI Modules.

HuggingFace

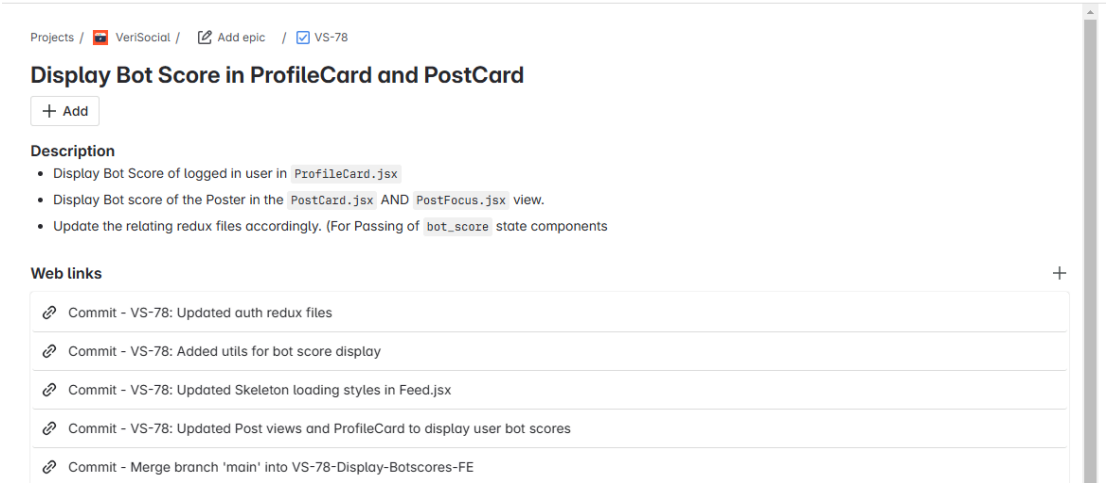
Purpose: HuggingFace provides pre-train models for AI-content detection for both images and text. We combined 1 model for each so that our platform can analyse scores containing combinations of text and images. Our approach is to treat these models as a **black-box** and use them as a way to combat the **Cold-Start** Problem.

Postman

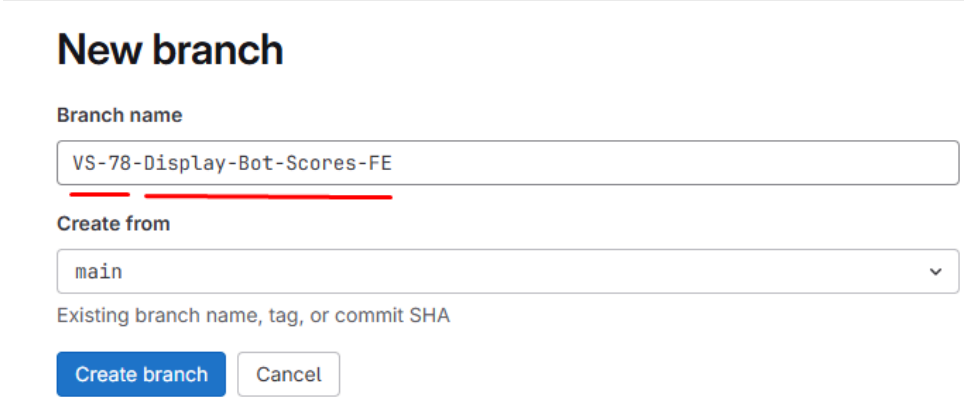
Purpose: A software application that allows developers to test, document and share APIs in a user friendly interface. For this project both the frontend and backend developers utilise Postman to test API endpoints before developing and pushing code.

The following illustrations show a simplified version of our typical development workflow. Starting from Jira with the Ticket creation and all the way to the end for Code Review before merging to the main branch.

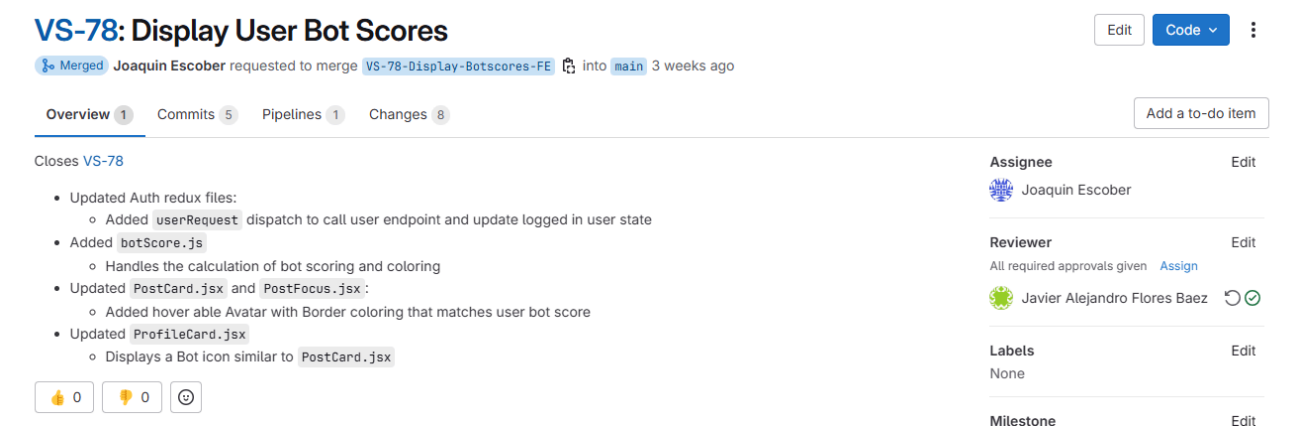
1. Feature Ticket Creation in Atlassian Jira.



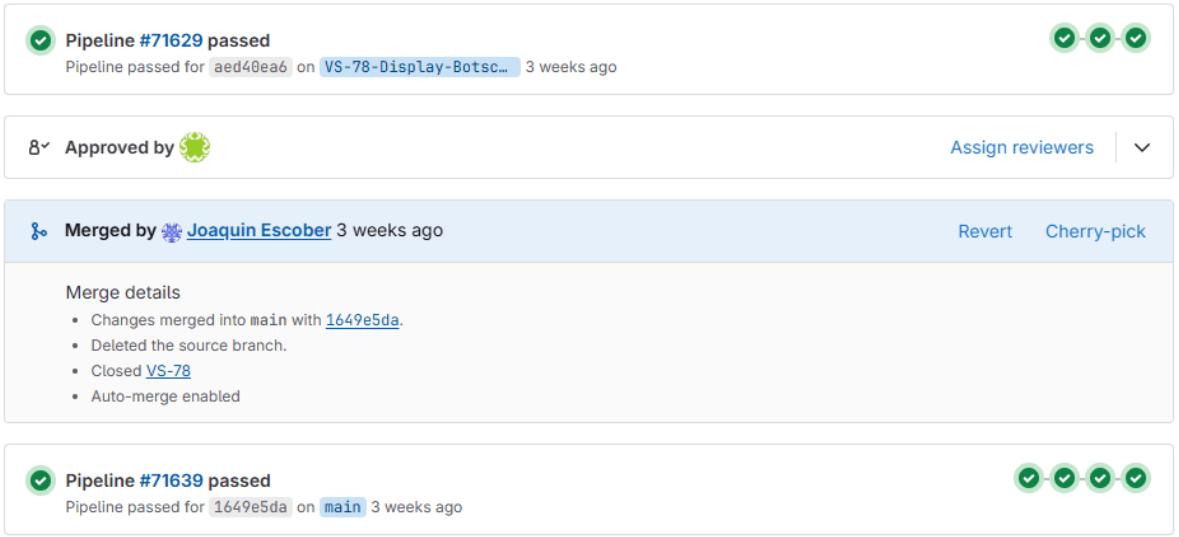
2. Create a new feature branch relating to the feature ticket. (In the form of VS-XX-feature-details).



3. Once code is implemented, tested and pushed by the assigned developer - we create a merge request.



4. Once all checks pass.(Approved by Reviewer and Pipeline passes). We can merge to main.



Once the feature branch merges to main. The feature ticket is automatically closed in Jira.

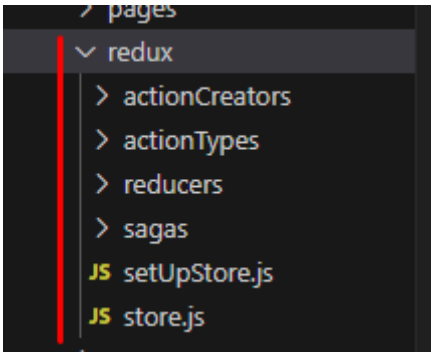
Sample Code & Walkthrough — Section 5

Key Code Snippets - (Frontend) — 5.1

Frontend Redux sample walkthrough

The frontend functionality revolves around the components accessing a globally shared memory state called the "store". Components in the frontend would typically access both their own local values (states) and the shared states in the store in order to perform their functions. Below is sample key code from our project that shows how the redux process works more in-depthly. For this example we will use the component LeaderboardCard.jsx to showcase how our redux system works.

The Redux files can be found in **../verisocial_fe/src/redux**. Below are the main directories that represent each stage of the redux process. The Redux process typically begins with declaring an **ActionType** -> Dispatching an **ActionCreator** -> **Middleware Saga** -> updating states in the **Reducer** in the Store.



In the **gameActionTypes.js** file, we can see all the actions relating to the game aspect of VeriSocial. For the Leaderboard component we are interested in the following Action Types. These are used to represent each type of action that will be dispatched for the middleware to handle. For each feature of the frontend, e.g. Loading a List of people on the Leaderboard, we define 3 ActionTypes. The Requesting Action (Calling API). The Success Action (Updating the Store upon successful Request) and finally the Failure Action (When the action or API call fails).

The Dispatcher of these actions will be handled by its own individual Action Creator. Creators define the function that will be used by Components and any parameters that need to be passed for the action itself. The **gameCreators.js** file showcases the **loadLeaderboardListRequest()** function to be used by the component along with its Success variant that shows that "payload" or data to be passed to the store for updating.

```
export const GAME_LOAD_LEADERBOARD_REQUEST = "GAME_LOAD_LEADERBOARD_REQUEST";
export const GAME_LOAD_LEADERBOARD_SUCCESS = "GAME_LOAD_LEADERBOARD_SUCCESS";
export const GAME_LOAD_LEADERBOARD_FAILURE = "GAME_LOAD_LEADERBOARD_FAILURE";
```

```
export const loadLeaderboardListRequest = () => ({
  type: GAME_LOAD_LEADERBOARD_REQUEST,
});

export const loadLeaderboardListSuccess = (leaderboardList) => ({
  type: GAME_LOAD_LEADERBOARD_SUCCESS,
  payload: { leaderboardList },
});
```

The saga acts as a middleware that watches each declared action and performs specific processes depending on which action is declared. This is usually where all the API calls are handled as redux-saga allows us to easily employ an asynchronous system. Below we can see the saga function for handling the retrieval of the leaderboard list.

```
function* handleLoadLeaderboadList() {
  try {
    const response = yield call(api.gameService.getLeaderboard);
    if (response.status === 'success' && Array.isArray(response.message)) {
      yield put(loadLeaderboardListSuccess(response.message));
    } else {
      yield put(loadLeaderboardListFailure(response.message || 'Failed to load leaderboard')
      );
    }
  } catch (err) {
    yield put(loadLeaderboardListFailure(err.message));
  }
}
```

```
export function* watchGameSaga() {
  yield takeLatest(GAME_EVENT_ADD_SCORE, handleAddGameEventScore);
  yield takeLatest(GAME_EVENT_LOAD_LIST_REQUEST, handleLoadGameEventList);
  yield takeLatest(GAME_LOAD_LEADERBOARD_REQUEST, handleLoadLeaderboadList);
}
```

We can see that the game saga monitors the game request actions, including the loading leaderboard example. In which case if it finds an instance of that action type being dispatched, it fires the **handleLoadLeaderboardList()** function. This handler function either succeeds or fails based on the response of the api call. In which case it dispatches the success or failure action using the creator variants of that action. In this case **loadLeaderBoardListSuccess()** or **loadLeaderboardListFailure()**.

The Reducer stage takes those actions dispatched, and performs an update on its store. The following image is taken from the gameReducer which updates game related states. It shows one of the many switch cases which performs a state update when detecting the appropriate dispatched action types. In this case a successful leaderboard list load along with its passed payload of the leaderboard list from the api call.

```
case GAME_LOAD_LEADERBOARD_SUCCESS: {
  const { leaderboard, user_rank } = action.payload.leaderboardList;
  return {
    ...state,
    loadingLeaderboard: false,
    leaderboard,
    userRank: user_rank,
  };
}
```

Now with the newly updated leaderboard list state - components will be able to access the most recently written dataset for the leaderboard list. It is accessed via connecting the component to the redux store. Below is code found in the LeaderboardListCard.jsx component. Now this component will be able to access the leaderboardList data without having to specify its own local state list.

```
const mapStateToProps = (state) => {
  const data = state.game.errorLeaderboard || {};
  return {
    leaderboardList: Array.isArray(data.leaderboard) ? data.leaderboard : [],
    userRank: data.user_rank || null,
  };
};
```

Key Code Snippets - (Backend) 5.2

The backend component uses the Actix-Web framework in order to handle requests and process the data. The main function is in charge of receiving all of the configuration for the code. It spawns a new app, it clones into it new web data that will be used safely between threads and requests. This is because web data in actix is wrapped by an `ARC` which is an atomic reference counter, which ensures that data is being shared safely.

```rust
HttpServer::new(move || {
    App::new()
        .wrap(Logger::default())
        .wrap(
            Cors::default()
                .allow_any_origin()  // Deployment inside docker compose
                .allowed_methods(vec!["GET", "POST", "PUT", "DELETE"])
                .allowed_headers(vec![header::AUTHORIZATION, header::CONTENT_TYPE])
                .supports_credentials()
                .max_age(3600)
        )
        .app_data(actix_web::web::PayloadConfig::new(usize::MAX))
        .app_data(user_repo.clone())
        .app_data(post_repo.clone())
        .app_data(comment_repo.clone())
        .app_data(game_event_repo.clone())
        .app_data(likes_repo.clone())
        .app_data(report_repo.clone())
        .app_data(web::Data::new(app_data.clone()))
        .service(routes::teapot::teapot)
        .configure(auth_config::config)
        .configure(user_config::config)
        .configure(post_config::post_config)
        .configure(post_config::game_config)
        .configure(post_config::admin_config)
})
.bind((app_url, 8080))?
.run()
.await
```

Aside from this the backend also utilizes the repository design pattern in order to abstract data retrieval layer and separate it from our data handling logic. Below you can see how this is implemented in Rust with the use of interfaces. These interfaces allow the application to mock the data that would be usually retrieved by these endpoints. Note that, there was a need to add the `async_trait` in order to make each method inside the repository asynchronous.

```rust
2 implementations
pub struct UserRepo {}
impl UserRepo {
    pub fn new() -> Self {
        Self {}
    }
}

#[automock]
#[async_trait]
4 implementations
pub trait UserTrait: Send + Sync {
    async fn create(&self, conn: &MySqlPool, new_user: NewUser, password: String) -> Result<u64, ApiError>;

    async fn find_all(&self, conn: &MySqlPool) -> Result<Vec<User>, ApiError>;

    async fn find_user_with_id(&self, conn: &MySqlPool, user_id: u64) -> Result<User, ApiError>;

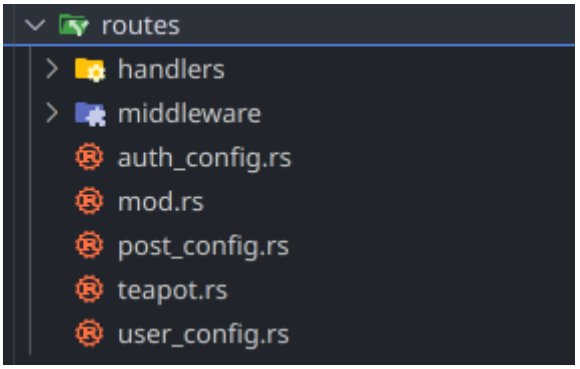
    async fn find_user_with_email(&self, conn: &MySqlPool, email: String) -> Result<User, ApiError>;
}


#[async_trait]
impl UserTrait for UserRepo {
    // Returns ID for the created user
    async fn create(&self, conn: &MySqlPool, new_user: NewUser, password: String) -> Result<u64, ApiError> {
        let result: Result<MySqlQueryResult, _> = sqlx::query_as!(User,
            r#"
            INSERT INTO users (username, email, password_hash, full_name, bio, bot_score)
            VALUES (?, ?, ?, ?, ?, ?)
            "#,
            &new_user.username,
            &new_user.email,
            password,
            &new_user.full_name,
            &new_user.bio,
            &new_user.bot_score
        )
        .execute(conn)
        .await;

        match result {
            Ok(query: MySqlQueryResult) => Ok(query.last_insert_id()),
            Err(err: Error) => Err(ApiError::CannotCreateObject(err.to_string()))
        }
    }
```

The routes in the application are also segregated into different configurations. These configurations allow for the modification of URIs and thus aggregating similar routes into each is recommended. Aside from that, each route needs to be explicitly defined in the **handlers** directory. Here is all of the code for the different routes. Finally, a middleware was added in order to add simple authentication capabilities to the application. This

authentication is performed through JWT encrypted with the HS256 algorithm to provide a certain level of security for users trying to access our services.



All the different data sent and received in the application is handled by models and the crate `serde` whose job is to deserialize and serialize all of the data from JSON to our data models defined in the models directory. Most models are defining either incoming data, data being stored in the database or data being sent back to the frontend. However, the more important model is the **response.rs** file that defines a more abstract model for all the responses that the backend sends. This was done with the use of generics, the **Response** model takes a generic variable **T** which can be any object that incorporates the trait **Serialize**. The trait only expects the object to have a method that serializes the object into JSON. Following that **Response** model there is the **ApiError** model which matches all of the errors in the backend into a response object which allows all endpoints to finish gracefully even if there was an error.

```rust
#[derive(Deserialize, Serialize)]
3 implementations
pub struct Response<T: Serialize> {
    pub status: String,
    pub message: T,
}

#[derive(Deserialize, Serialize)]
3 implementations
pub enum ApiError {
    ServerError(String),
    CannotCreateObject(String),
    ObjectNotFound(String),
    UpdateError(String),
    DeleteError(String),
    AuthError(String),
    CannotUpload(String)
}


impl<T> Response<T>
where T: Serialize
{
    pub fn create_post_response(data: Result<T, ApiError>) -> HttpResponse {
        match data {
            Ok(user: T) => {
                HttpResponse::Created().json(Response {
                    status: "success".into(),
                    message: user
                })
            },
            Err(err: ApiError) => Self::create_error_response(err)
        }
    }

    pub fn create_get_response(data: Result<T, ApiError>) -> HttpResponse {
        match data {
            Ok(a: T) => HttpResponse::Ok().json(Response {
                status: "success".into(),
                message: a,
            }),
            Err(err: ApiError) => Self::create_error_response(err)
        }
    }

    pub fn create_error_response(err: ApiError) -> HttpResponse {
        match err {
            ApiError::ServerError(description: String) => HttpResponse::InternalServerError().json(Response{
                status: "failed".into(),
                message: "Internal Server Error: ".to_owned() + &description.to_owned()
            }),
            ApiError::CannotCreateObject(description: String) => HttpResponse::BadRequest().json(Response {
                status: "failed".into(),
                message: description
            }),
```

Other than this there is more code that is used as utilities such as config and services. The config directory is where the initialization of the configuration in the application starts, and the services are there for more validation. The validation it provides is minimal and checks for things such as (e.g, if the user is in the database already or not).

The AI Content Detection is a server built with Python using the sentence-transformers library in order to classify the different content our main application produces. This server has only one endpoint called classify where it receives only one type of data. On each call the **RequestBody** is added to an asynchronous queue, which then a separate worker thread grabs the post and processes it. The worker also has other functions such as updating the score of whatever post was sent to it. The worker itself logs if something went wrong and prints the traceback, giving further information as to why that happened without shutting down the queue.

```python
class RequestBody(BaseModel):
    post_id: int
    text: str
    image_url: Optional[str]

class ProcessedPost(BaseModel):
    post_id: int
    text_score: float
    image_score: float | None
    total_score: float
```

```python
async def classifier_worker():
    loop = asyncio.get_running_loop()
    while True:
        try:
            req = await queue.get()
        except asyncio.CancelledError:
            break

        try:
            img = None
            if req.image_url:
                img = await get_image(req.image_url)
                result = await loop.run_in_executor(None, classify_text_and_image, req, img)
            else:
                result = await loop.run_in_executor(None, classify_text, req)

            await update_post_score(result)
        except Exception:
            print(f"Error processing {req.post_id}")
            traceback.print_exc()
        finally:
            queue.task_done()
```

As for the proper classification of the content, it happens on the **classifier.py** file. Here the pipelines for the different classification models are brought into scope and initialized. These are then used in the subsequent functions **classify_text_and_image** and **classify_text**. Below there is a quick example of how the code works. Note that our code uses simple weighted averages in order to determine a total_bot_score for a post. Since we receive floats from the classification pipelines, we use a weight that's between 0 and 1 in order to find the proper score. The weight is applied to the image score first, and then `0 - weight` is applied to the text score. There is one final important thing to mention.

Most of the classifier models give you a label and a score, the label is the classification and the score is their confidence. Our project uses the confidence to identify the bot score, but it leaves the awkward situation where a label is either “Human” or “Real”, and then it gives a high score for that post. That is why in the code statements such as `1 - image_score` are there.

TOTAL_SCORE = (IMAGE_SCORE * WEIGHT) + (TEXT_SCORE * (1 - WEIGHT))

```python
image_classifier = pipeline(
    "image-classification",
    model="NYUAD-ComNets/AI-generated_images_detector",
    device="cpu"
)
text_classifier = pipeline(
    "text-classification",
    model="fakespot-ai/roberta-base-ai-text-detection-v1",
    device="cpu"
)


def classify_text_and_image(body: RequestBody, image) -> ProcessedPost:
    # Write the AI classification function
    image_pred = image_classifier(image)
    text_pred = text_classifier(body.text)

    # Create Response
    text_class_type = text_pred[0]["label"]
    text_score = text_pred[0]["score"]

    #Create Response (needs to be changed to be custom, or maybe not?)
    image_class_type = image_pred[0]["label"]
    image_score = image_pred[0]["score"]

    weight = settings.weight

    # Set a weighted average for total score
    if text_class_type == "Human" and image_class_type == "real":
        total_score = ((1 - image_score) * weight) + ((1 - text_score) * (1 - weight))

        response = ProcessedPost(post_id=body.post_id, text_score=1 - text_score, image_score=1 - image_score, total_score=total_score)
    elif text_class_type == "Human" and image_class_type != "real":
        total_score = (image_score * weight) + ((1 - text_score) * (1 - weight))

        response = ProcessedPost(post_id=body.post_id,  text_score=1 - text_score, image_score=image_score, total_score=total_score)
    elif text_class_type != "Human" and image_class_type == "real":
        total_score = ((1 - image_score) * weight) + (text_score * (1 - weight))

        response = ProcessedPost(post_id=body.post_id,  text_score=text_score, image_score=1 - image_score, total_score=total_score)
    else:
        total_score = (image_score * weight) + (text_score * (1 - weight))
        response = ProcessedPost(post_id=body.post_id, text_score=text_score, image_score=image_score, total_score=total_score)

    return response
```

Key Code Snippets - (Deployment) 5.4

For deployment our project uses a mix between docker-compose and system services. More on why we use system services instead of an all docker environment later in the problems solved section.

The main services run on a docker-compose that is in charge of orchestrating these. These services are the frontend and the backend. We use the command `**docker compose pull**` to pull for changes in the docker service, and we use `**docker compose up -d**` for the services to run detached. Note that in order to achieve this, we SSH-ed into our digitalOcean droplet and then cloned the git repo.

```
root@dcu-fyp-jajo-dnubuntu2204-s-1vcpu-1gb-lon1-01:~# docker ps
CONTAINER ID   IMAGE                                  COMMAND                  CREATED          STATUS          PORTS                                        NAMES
c4a41830a5f9   cr0c0dile07/final-year-project:frontend   "/docker-entrypoint.…"   52 minutes ago   Up 52 minutes   0.0.0.0:80->80/tcp, :::80->80/tcp            frontend
1d5df8f4dd4c   cr0c0dile07/final-year-project:backend    "./verisocial_be"        52 minutes ago   Up 52 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp    backend
root@dcu-fyp-jajo-dnubuntu2204-s-1vcpu-1gb-lon1-01:~#
```

The other services running the bot detection and the ai content detection, they run as system services due to their final docker images being very heavy. Below is the system service description of one of our services that runs that way. Note that, since the **uv** python package manager was used in order to instantiate virtual environments for python, the service needs to be able to find the virtual environment.

```
[Unit]
Description=AI Content Checker
After=network.target

[Service]
User=your_user
WorkingDirectory=/root/2025-csc1097-escobej2-floresj2/src/ai_content_detection
ExecStart=/root/2025-csc1097-escobej2-floresj2/src/ai_content_detection/venv/bin/uvicorn main:app --reload --host 0.0.0.0 --port 8000 --env-file .env
Restart=always
Environment=PATH=/root/2025-csc1097-escobej2-floresj2/src/ai_content_detection/.venv/bin:/usr/bin:/bin
Environment=VIRTUAL_ENV=/root/2025-csc1097-escobej2-floresj2/src/ai_content_detection/.venv

[Install]
WantedBy=multi-user.target
```

```
● ai-content-checker.service - AI Content Checker
     Loaded: loaded (/etc/systemd/system/ai-content-checker.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-05-01 22:05:54 UTC; 17h ago
   Main PID: 163088 (uvicorn)
      Tasks: 5 (limit: 1113)
     Memory: 577.8M (peak: 751.1M)
        CPU: 12h 15min 50.206s
     CGroup: /system.slice/ai-content-checker.service
             ├─163088 /root/2025-csc1097-escobej2-floresj2/src/ai_content_detection/.venv/bin/python /root/2025-csc1097-escobej2-floresj2/src/ai_content_detection/.venv/bin/uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
             ├─163090 /root/2025-csc1097-escobej2-floresj2/src/ai_content_detection/.venv/bin/python -c "from multiprocessing.resource_tracker import main;main(6)"
             └─163131 /root/2025-csc1097-escobej2-floresj2/src/ai_content_detection/.venv/bin/python -c "from multiprocessing.spawn import spawn_main; spawn_main(tracker_fd=7, pipe_handle=9)" --multiprocessing-fork
```

Our system requires quite a bit of configuration variables to be set up. These are setup at the start of a deployment inside a **config.yml** file. The configuration is as follows:

```yaml
app_url: 'app_url'
app_database_url: 'database' # Database URL for rust backend
auth_secret: 'secret' # Generated secret for HS256 algorithm
ai_image_checker_url: 'image_checker_url'
max_file_size: 10485760 # max file size for rust backend

# For python server
database_user: "db_user"
database_host: "db_host"
database_port: "db_port"
database_secret: "db_secret"
database_name: "db_name"

# AWS config for digital ocean
aws_secret_access_key: "aws_secret"
aws_endpoint: "aws_endpoint"
aws_region: "region"
aws_access_key_id: "aws_access_key_id"

bucket_name: "bucket_name" # to store the images

rust_log: "debug"

# for ensemble.py (detector algorithm weighting)
anomaly:
  weights:
    one_class_svm: 0.5
    lof: 0.5
    isolation_forest: 2.0
  threshold: 0.0
# for updating bot score
update:
  bot_multiplier: 0.2 # what to multiply each pred by when updating bot_score
  human_multiplier: 0.01

time_interval:
  minutes: 5
```

All the different configurations do something for the system, and this file should be found at the root of the cloned repository in every project in order for the system to behave as intended.

Problems Solved Section 6

The following are some technical / design problems encountered during development and how we strategize in solving them.

Identified Issues 6.1

1. Using Isolation Forests by themselves

While Isolation Forests (**IFs**) are well-suited for anomaly detection, it does have its own weaknesses when used standalone. Particularly when the sample size (user count in this case) is low. Although many such anomaly detection algorithms suffer from this same reason, IFs are especially sensitive to this as they rely on the random partitioning of feature space to isolate data points. If the sample sample size is too small, the cut-off points when doing the random partitioning may not effectively differentiate between normal and anomalous points. Although our primary intention is to treat these bot detection (and the AI content detection) as **black-boxes** - it is important to note that this method alone can lead to poor model performance.

2. AI Deployment in Docker

The deployment within Docker of the AI modules was not ideal. Building these would take a long time as it would mean downloading heavy ML libraries for the models. This meant we could not put them through the pipeline to follow our CI/CD flow.

Strategies and Solutions 6.2

Multi-Model Approach

One way we mitigated the issue of using Isolation Forests by themselves is to adopt a multi-model / algorithm approach. Utilising the Scikit-learn package and their ready made anomaly detection algorithms, we can inject multiple ways of detecting anomalies out-of-the-box in our bot detection workflow. For this we chose the algorithms One-Class SVM and Local Outlier factor from Scikit-Learn.

- One-Class SVM works well even with smaller datasets AND with high-dimensional data. This helps cover for IFs particular weakness in smaller datasets. In addition One-Class SVM algorithms can be more robust, as the algorithm focuses on the distribution of “normal” data then classifies the rest as anomalies. In other words, unlike how IFs focus on finding anomalous data points, One-Class SVM is more focused on finding what's normal in the data set.
- For Local Outlier Factor (**LOF**), anomaly detection is measured with **variable densities**, (if a data point has significantly lower density than its neighbors, it is considered an outlier). IFs measure outliers through the **global space** (treats dataset as a whole), meaning that it doesn’t perform well when trying to find locally isolated outliers. LOF covers this weakness, because it is concerned with finding outliers in respect to its neighbours.
- IFs also struggle with high-dimensional / high-feature data, so in this case we decided to limit the dimensions of the user activity vector to 3 columns. (max count, standard deviation, and burst count).

With all these algorithms in mind, we wanted to incorporate a non-biased approach. To remove the Single-Model bias of the “Isolation Forest only” approach, a design decision was made to put the 3 algorithms into a combined ensemble. This method allows us to combine the strength of each algorithm, each compensating for their weaknesses. Combined with the ability to configure the weights of each algorithm, we can create a customisable bot-detection flow.

Deploy System Service

The solution we came up with was to deploy using system services. That way using virtual environment of python, each deployment for the AI modules did not need to download packages or libraries.

Results, Testing and Analysis Section 7

NOTE: Testing is covered more in-depthly in a separate Testing Document. Below is a high-level overview of the testing done and some of the analysis.

Testing Strategies 7.1

Unit Testing (Frontend)

To ensure reliability and functionality of our web application, we utilised unit tests. For the frontend, unit testing was performed on key components. React Testing Library and Vitest were used in conjunction so that we were able to test component UI behaviour and interactions.

- **React Testing Library**: Used to similar user interactions and test the results by verifying them with expected behaviour. This library allowed the use of rendering events for "mocking" user interactions. These included click, inputting and submitting forms.
- **Vitest**: A testing framework that provides fast test execution and integration with React Vite bundler.

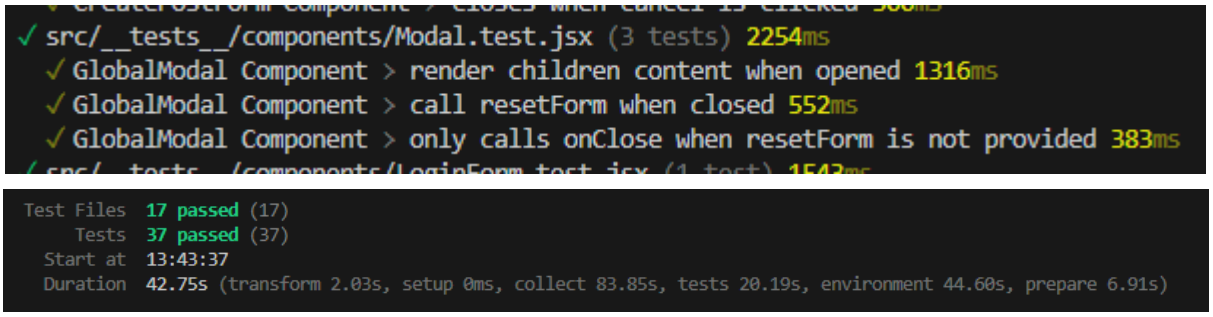


Fig.7.1 - Sample view of Unit Tests

Integration Testing (using PostMan)

Integration testing was done through PostMan. This was to verify that the requests were giving the proper data before and after being deployed. Inside PostMan we had two environments one that was for the deployed version of the system and another for the local version of the system.

The downside of these tests is that they were not able to be put into the pipeline and thus it was run by the developers very similar to **ad-hoc** testing. However, these had programmatic ways of ensuring that the data being returned was proper. Below, you can find one such test for the request.

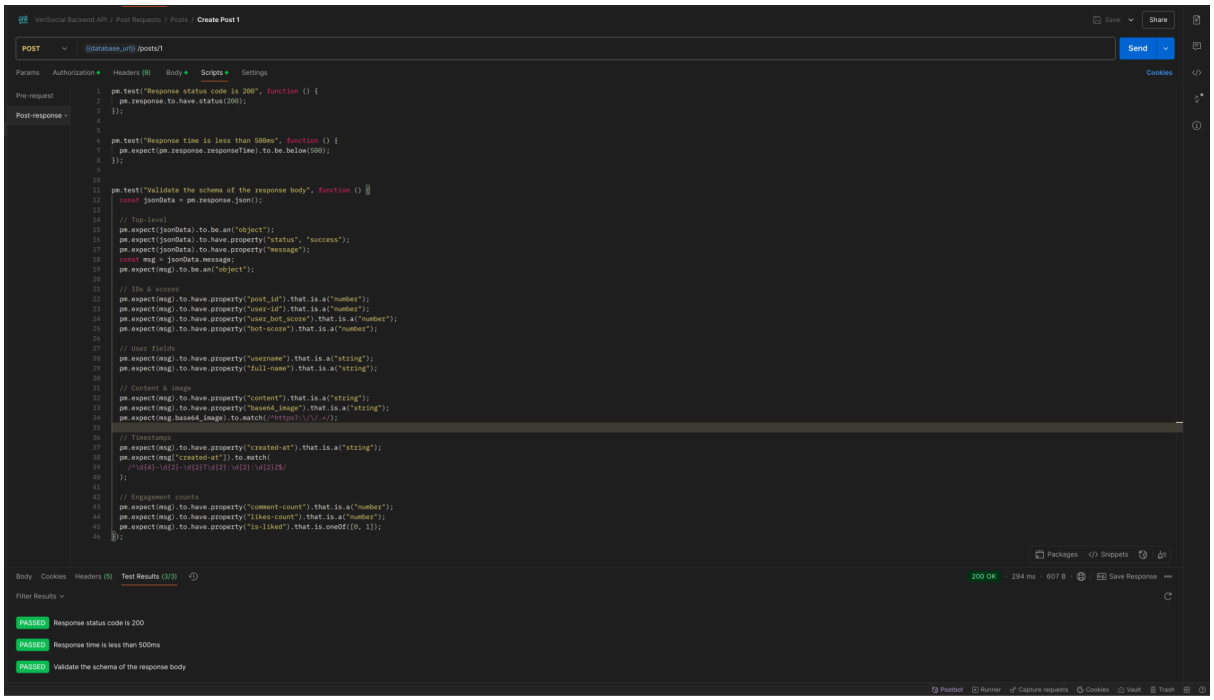


Fig.7.2 - Sample view of Integration Tests

Future Work — Section 8

Limitations — 8.1

Scalability:

As the platform grows and more users are registered to our platform, we may face issues regarding infrastructure and scalability. Currently for the requirements and scale of this project, there is no need to create a system that allows for thousands of concurrent active users. However handling large volumes of user data and API requests, particularly when scaling the AI and bot detection modules, could pose a challenge and may risk system health. Optimising becomes necessary in future for smooth operation, availability and reliability when the user base increases.

Limited Data Source:

Currently the Bot Detection relies on the entire user base to calculate predictions. User activity across the app is measured and stored for calculation, making our bot detection algorithms dependent on user count and their activity frequency. To get more accurate predictions, having high user count is important as we are able to capture more user activity vectors to compare when detecting bot behaviour. However it is also worth noting that the increase of user count also increases execution time for when we scan the user base for bots.

Potential Improvements — 8.2

1. Scoring based on content type

Currently a user's bot score correlates to the activity frequency. This includes count, average time between activity and the amount of times a user has displayed a burst of activity. One area we wanted to improve on

was measuring more than just the numerics of their activity. For example we can potentially categorise content they post into classes, e.g. spam, advertisement of inappropriate content. This could contribute or affect their bot score allowing us to score users in not just their activity levels but also type of posts.

2. Standardize model outputs

Currently, our model outputs are very specific to our models, however our intention is for the models to be plug and play with different models. We didn't want to create a system specific to a specific model but one that could handle different models.

Long-term Vision 8.3

Mobile Compatibility

While currently the web application is functioning and provides a way to showcase our initial project concept, we can also decide to add mobile compatibility for the VeriSocial platform. This will allow us to expand our audience to not only web users but also mobile users and increase our accessibility for the platform.